

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №10

по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Поиск расстояний во взвешенном графе»

Выполнили:
студенты группы 24ВВВЗ
Савин А.В.
Майер В.С.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., Деев М.В.

Пенза 2025

Цель работы – Освоение методов генерации, визуализации и анализа графов через создание программы, которая реализует генерацию матриц смежности для взвешенных ориентированных и неориентированных графов, выполняет поиск расстояний между вершинами с использованием очереди, вычисляет метрические характеристики (радиус, диаметр, центральные и периферийные вершины), а также обеспечивает обработку параметров командной строки для гибкой настройки типа графа.

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс `queue` из стандартной библиотеки C++.
- 3.* Сгенерируйте (используя генератор случайных чисел) матрицу смежности для ориентированного взвешенного графа G . Выведите матрицу на экран и осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием.

Задание 2

1. Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.
2. Определите подмножества периферийных и центральных вершин.

Задание 3*

1. Модернизируйте программу так, чтобы получить возможность запуска программы с параметрами командной строки (см. описание ниже). В качестве параметра должны указываться тип графа (взвешенный или нет) и наличие ориентации его ребер (есть ориентация или нет).

Код программы

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <conio.h>
#include <locale.h>
#include <queue>
#include <climits>

using namespace std;

void BFSF(int** G, int numG, int** GD, int s, int isDirected) {
    queue<int> q;
    int v;
    int* distance = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        distance[i] = INT_MAX;
    }

    q.push(s);
    distance[s] = 0;

    while (!q.empty()) {
        v = q.front();
        q.pop();

        for (int i = 0; i < numG; i++) {
            if (G[v][i] > 0 && distance[i] > distance[v] + G[v][i]) {
                q.push(i);
                distance[i] = distance[v] + G[v][i];
            }

            if (!isDirected && G[i][v] > 0 && distance[i] > distance[v] + G[i][v])
            {
                q.push(i);
                distance[i] = distance[v] + G[i][v];
            }
        }
    }

    for (int i = 0; i < numG; i++) {
        if (distance[i] == INT_MAX) {
            GD[s][i] = -1;
        }
        else {
            GD[s][i] = distance[i];
        }
    }
}
```

```

    }
}

free(distance);
}

void printM(int** Matr, int numG, const char* name) {
    printf("\n%s (%d x %d):\n", name, numG, numG);
    printf("    ");
    for (int j = 0; j < numG; j++) {
        printf("%3d ", j);
    }
    printf("\n");

    for (int i = 0; i < numG; i++) {
        printf("%2d: ", i);
        for (int j = 0; j < numG; j++) {
            if (Matr[i][j] == -1) {
                printf(" - ");
            }
            else {
                printf("%3d ", Matr[i][j]);
            }
        }
        printf("\n");
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    int** G;
    int** GD;
    int* ecc;
    int numG, current;
    int graphType;

    printf("Введите количество вершин: ");
    scanf("%d", &numG);

    if (numG <= 0) {
        printf("Ошибка: количество вершин должно быть положительным!\n");
        return 1;
    }

    printf("Выберите тип графа:\n");
    printf("0 - Неориентированный граф\n");
    printf("1 - Ориентированный граф\n");
    printf("Ваш выбор: ");
    scanf("%d", &graphType);

```

```

if (graphType != 0 && graphType != 1) {
    printf("Ошибка: неверный тип графа!\n");
    return 1;
}

ecc = (int*)malloc(numG * sizeof(int));
G = (int**)malloc(numG * sizeof(int*));
GD = (int**)malloc(numG * sizeof(int*));

if (!ecc || !G || !GD) {
    printf("Ошибка выделения памяти!\n");
    return 1;
}

for (int i = 0; i < numG; i++) {
    G[i] = (int*)malloc(numG * sizeof(int));
    GD[i] = (int*)malloc(numG * sizeof(int));
    if (!G[i] || !GD[i]) {
        printf("Ошибка выделения памяти!\n");
        return 1;
    }
}

printf("\nГенерация %s графа...\n",
    graphType == 0 ? "неориентированного" : "ориентированного");

for (int i = 0; i < numG; i++) {
    ecc[i] = 0;
    for (int j = 0; j < numG; j++) {
        if (i == j) {
            G[i][j] = 0;
        }
        else {
            if (graphType == 0) {
                if (j > i) {
                    if (rand() % 2 == 1) {
                        int weight = (rand() % 10) + 1;
                        G[i][j] = weight;
                        G[j][i] = weight;
                    }
                    else {
                        G[i][j] = 0;
                        G[j][i] = 0;
                    }
                }
            }
            else {
                if (rand() % 2 == 1) {
                    G[i][j] = (rand() % 10) + 1;
                }
            }
        }
    }
}

```

```

        else {
            G[i][j] = 0;
        }
    }
}
GD[i][j] = 0;
}
}

printM(G, numG, "Матрица смежности");

printf("\nВычисление матрицы расстояний...\n");
for (int i = 0; i < numG; i++) {
    BFS(D, G, numG, GD, i, graphType);
}
printM(GD, numG, "Матрица расстояний");

for (int i = 0; i < numG; i++) {
    ecc[i] = 0;
    for (int j = 0; j < numG; j++) {
        if (i != j && GD[i][j] > 0 && GD[i][j] != -1) {
            if (GD[i][j] > ecc[i]) {
                ecc[i] = GD[i][j];
            }
        }
    }
}

printf("\nЭксцентриситеты вершин:\n");
int hasValidEccentricities = 0;
for (int i = 0; i < numG; i++) {
    if (ecc[i] > 0) {
        printf("Вершина %d: %d\n", i, ecc[i]);
        hasValidEccentricities = 1;
    }
    else {
        printf("Вершина %d: недостижима или изолирована\n", i);
    }
}

int radius = INT_MAX;
int diameter = 0;

for (int i = 0; i < numG; i++) {
    if (ecc[i] > 0) {
        if (ecc[i] < radius) {
            radius = ecc[i];
        }
        if (ecc[i] > diameter) {
            diameter = ecc[i];
        }
    }
}

```

```

    }
}

if (radius == INT_MAX) {
    printf("\nГраф не имеет достижимых вершин или все вершины изолированы.\n");
}
else {
    printf("\nРадиус графа: %d\n", radius);
    printf("Диаметр графа: %d\n", diameter);

    printf("\nЦентральные вершины (эксцентриситет = радиусу = %d): ", radius);
    int centralFound = 0;
    for (int i = 0; i < numG; i++) {
        if (ecc[i] == radius) {
            printf("%d ", i);
            centralFound = 1;
        }
    }
    if (!centralFound) printf("нет");

    printf("\nПериферийные вершины (эксцентриситет = диаметру = %d): ",
diameter);
    int peripheralFound = 0;
    for (int i = 0; i < numG; i++) {
        if (ecc[i] == diameter) {
            printf("%d ", i);
            peripheralFound = 1;
        }
    }
    if (!peripheralFound) printf("нет");
}

if (graphType == 1) {
    int isSymmetric = 1;
    for (int i = 0; i < numG && isSymmetric; i++) {
        for (int j = i + 1; j < numG; j++) {
            if (G[i][j] != G[j][i]) {
                isSymmetric = 0;
                break;
            }
        }
    }

    if (isSymmetric) {
        printf("Граф симметричный (по факту неориентированный)\n");
    }
    else {
        printf("Граф асимметричный (ориентированный)\n");
    }

    printf("\nПолустепени вершин:\n");
    for (int i = 0; i < numG; i++) {

```

```

        int outDegree = 0;
        int inDegree = 0;

        for (int j = 0; j < numG; j++) {
            if (G[i][j] > 0) outDegree++;
            if (G[j][i] > 0) inDegree++;
        }

        printf("Вершина %d: исходящая степень = %d, входящая степень = %d\n",
               i, outDegree, inDegree);
    }
}

for (int i = 0; i < numG; i++) {
    free(G[i]);
    free(GD[i]);
}
free(G);
free(GD);
free(ecc);

_getch();
return 0;
}

```

Результаты работы программы

```

Введите количество вершин: 10
Выберите тип графа:
0 - Неориентированный граф
1 - Ориентированный граф
Ваш выбор: 1

Генерация ориентированного графа...

Матрица смежности (10 x 10):
    0   1   2   3   4   5   6   7   8   9
0:   0   0   0   0   8   4   0   2   0   0
1:   9   0   8   0   8   0   0   5   0  10
2:   2   2   0   8   0   1   5  10   0   7
3:   0   6   0   0   0   6   5   5   0   0
4:   2   0   5   9   0   0  10   7   0   8
5:   9   9   8   0   7   0   2   9   6   6
6:   0   0   1   0   0   2   0   0   1  10
7:   0   0   1   6   0   9   2   0   0   0
8:   0   0   0   0   6   4   3   6   0   4
9:   0   0   1   0   0   2   4   0   7   0

Вычисление матрицы расстояний...

Матрица расстояний (10 x 10):
    0   1   2   3   4   5   6   7   8   9
0:   0   5   3   8   8   4   4   2   5   9
1:   8   0   6  11   8   7   7   5   8  10
2:   2   2   0   8   8   1   3   4   4   7
3:   8   6   6   0  12   6   5   5   6  10
4:   2   7   5   9   0   6   6   4   7   8

```

Рисунок 1 - Результат работы программы


```
Эксцентриситеты вершин:
Вершина 0: 9
Вершина 1: 11
Вершина 2: 8
Вершина 3: 12
Вершина 4: 9
Вершина 5: 11
Вершина 6: 9
Вершина 7: 9
Вершина 8: 12
Вершина 9: 9

Радиус графа: 8
Диаметр графа: 12

Центральные вершины (эксцентриситет = радиусу = 8): 2
Периферийные вершины (эксцентриситет = диаметру = 12): 3 8 Граф асимметричный (ориентированный)

Полустепени вершин:
Вершина 0: исходящая степень = 3, входящая степень = 4
Вершина 1: исходящая степень = 5, входящая степень = 3
Вершина 2: исходящая степень = 7, входящая степень = 6
Вершина 3: исходящая степень = 4, входящая степень = 3
Вершина 4: исходящая степень = 6, входящая степень = 4
Вершина 5: исходящая степень = 8, входящая степень = 7
Вершина 6: исходящая степень = 4, входящая степень = 7
Вершина 7: исходящая степень = 4, входящая степень = 7
Вершина 8: исходящая степень = 5, входящая степень = 3
Вершина 9: исходящая степень = 4, входящая степень = 6
```

Рисунок 2 - Результат работы программы

Вывод: В ходе лабораторной работы были успешно освоены методы генерации, визуализации и анализа графов, реализованы алгоритмы поиска расстояний с использованием очереди, вычислены метрические характеристики графов.